Low-cost coding agents in survival Minecraft: a tested skill library, a twelve-day run of four Gemini 3.8 Flash agents, and a server-log audit of every milestone

Oleksandr Pylypenko

Independent Researcher in Artificial Intelligence and Machine Learning

PhD Student, State University of Trade and Economics, Kyiv, Ukraine

ORCID: 0009-0004-9610-9242

O.Pylypenko@knute.edu.ua

2026-11-03

Abstract

We partially reproduce the setting of arXiv:2610.22319, in which four Gemini 4 Argon agents beat Minecraft by writing JavaScript programs for the Mineflayer library. Our setting differs in a cheaper model (Gemini 3.8 Flash High through the Antigravity CLI), a limited token budget, and a tested skill library given to the agents at the start: 54 functions, among them a recursive `lib.obtain(item, n)` that derives the whole chain of prerequisites from the game data. We describe the infrastructure (a sandboxed bridge, a turn watchdog, "parking" of idle bots, a cutaway viewer), 20 defects found while testing it, two pilot runs and a main run of 294 calendar hours, 20.1 of them active play. In that time 2–4 agents ran 3770 programs (21.2 % failed; 70.4 % of the failures were time-outs) and died 332 times. The main contribution is the audit: each of 16 milestones receives an evidence level A–D by a mechanical rule over the server log. Level A means that no console command at all preceded the milestone. The agents completed stage 1 (stone, iron, armour, bed, diamonds, diamond pickaxe, flint and steel, obsidian) at level A in 2.4 active hours. Entering the Nether, finding a fortress and the first blaze rod are level B (the only help was night skipping), but by then the agents were using helpers that the operator's supervising session had added during the run (mark +K). Ender pearls and eyes of ender are level C (permanent status effects, enchantments, `keepInventory`). We do not credit the agents with the stronghold, the End or the dragon: console commands produced those milestones (teleports, `/fill` of the portal, `/damage` to the dragon, an advancement grant), level D. Fire, ghast fireballs and lava account for 63.6 % of deaths. The model used 13 652 steps, 50.8 M uncached input tokens (1922.9 M more from the cache, 97.4 %), 8.82 M output and 4.06 M reasoning tokens.

Keywords: LLM agents; multi-agent systems; Minecraft; Mineflayer; skill library; long horizon; audit; reproducibility.

Evidence scope. Levels A–D are assigned mechanically from the server log, without manual judgement; agent behaviour is described from the chat logs; cost is reported in tokens, not money. All numbers come from one main run and two pilots. Level-D milestones are not agent results and are listed only to complete the audit.

Independence statement. This is an independent partial reproduction of the setting of arXiv:2610.22319 (Google DeepMind). As of 2026-11-02 there was no contact with its authors; the work was not coordinated with or reviewed by them. We do not use Gemini 4 Argon and do not claim our results are directly comparable to theirs.

1 Introduction and motivation

In arXiv:2610.22319 [1] four Gemini 4 Argon agents went from bare hands to defeating the Ender Dragon in 27 h 14 min, running 2783 programs and dying 605 times. They had no game-specific tools: only Mineflayer [5], its path-finding plugin and a one-paragraph goal. The authors estimate the run at about $520 at introductory list prices.

At the other end, Odyssey [2] gives agents a large skill library (40 primitive and 183 compositional skills), which lets even an 8-billion-parameter model solve tasks such as "obtain a diamond"; without the library, in the authors' ablation, such a model hardly produces executable code. Voyager [3] grows its own library with GPT-4.

We asked an intermediate question: how far do cheaper coding agents get when they receive a tested skill library and guides but keep full freedom of code and coordination, and how should one report a run into which the operator stepped under budget pressure? The second question turned out to matter as much as the first: near the end of our run, the operator's supervising session "finished" the game with console commands at the author's request, and without an audit against the server log this part would read as the agents' result.

2 What is checked and what is out of scope

Checked: how far along the Minecraft technology tree 2–4 Gemini 3.8 Flash High agents with a skill library, a watchdog and guides get, and in how much active time; which failures (deaths, program errors, infrastructure defects) set the pace; whether what the agents achieved can be separated, mechanically and from the server log, from what operator commands did.

Out of scope: a controlled comparison of models (one run, one model); a dragon kill by the agents (not achieved, Section 6.2); playing from pixels (agents see the world only through the Mineflayer API); judging the guides separately from the library (they changed together).

3 Contributions, and what they do not claim

(a) **Open infrastructure**: a bridge to Mineflayer with a `vm` sandbox and an allow-list of modules, the `./mc` command for agents, a turn watchdog for `agy`, parking of idle bots, protection of placed blocks from the path finder, an overhead cutaway viewer. *Not claimed:* that it is optimal or works unchanged on other game versions.

(b) **A skill library** of 54 functions, including `lib.obtain`, which derives the chain of prerequisites from game data (recipes, block drops, harvest tools, smelting, mob drops). Every skill was tested on the server; the defects found are listed in Table 7. *Not claimed:* completeness; combat, the Nether and the End are covered weakly.

(c) **Results** of two pilots and a main run, with evidence levels A–D for 16 milestones (Table 2). *Not claimed:* that the agents beat the game; milestones M14–M16 are level D.

(d) **An audit method**: a mechanical rule over the server log, and a separate +K mark for helpers the operator added during the run. *Not claimed:* that it detects every form of help; it sees changes to the instructions and advice only through the agents' use of helpers.

(e) **A failure analysis**: 332 deaths by cause and phase, 798 failed programs by class. *Not claimed:* a causal link between assistance phase and death rate; the phases coincide with the move into the Nether.

(f) **Token accounting** for the main run. *Not claimed:* a monetary cost; quota access through Antigravity has no transparent per-token price.

4 Environment and agents

Server. Vanilla Minecraft Java Edition 1.20.1 in Docker (`itzg/minecraft-server`), survival, Easy, no cheats, command blocks or RCON; the only operator is the human spectator. We started on 1.20.4, as arXiv:2610.22319 did, but moved to 1.20.1 because the 3D viewer prismarine-viewer ships the block-state palette of exactly that version and drew underground blocks wrongly on 1.20.4 (defect D12).

Bridge. Each player has its own Node.js process with a Mineflayer bot. An agent writes a program to a file and runs `./mc run file --timeout N`. The program runs as the body of an `async` function in a `vm` sandbox; its globals are `bot`, `lib`, `console`, `isCancelled()`, `Vec3`, `mcData`, `goals`, `Movements`, `sleep`. Only the Mineflayer packages and a few harmless Node modules can be required; processes, files and the network are unavailable. Values on `globalThis` persist between programs, and the team's and the agent's `autoload.js` run after every bridge start. After a time-out `isCancelled()` becomes true and movement and digging stop. The output of a program is followed by position, health, food, phase of day, nearby hostiles, tool durability and new chat. After 5 minutes without programs the bot logs out ("parking") and comes back with the next program; chat and events meanwhile are read from the server log.

Skill library. 54 functions (`bridge/lib/skills.js` and `obtain.js`, 1442 lines at the end of the run): looking around, movement (digging allowed, but never through blocks the agents placed), gathering, crafting with recovery from failures, placing and blueprints (house, portal frame), chests, smelting, combat, shelter, sleep, a staircase down to diamonds, buckets, obsidian from lava and water, doors. In tests from an empty inventory an iron pickaxe took 96 s, a bucket 99 s and full iron armour about 6 min.

How `lib.obtain` derives a chain. `lib.obtain(item, n)` makes sure at least n items are in the inventory and for each missing item picks a source in this order. (1) Smelting, if the item is in the smelting table (ingots, glass, cooked meat, charcoal): first the input, then fuel at 8 items per coal or 1.5 per plank or log, and only right before smelting it finds or places a furnace. (2) Crafting: among all recipes the cheapest by an estimate of ingredient cost is chosen; recipes that consume storage blocks are dropped and recipes from the same item family (a bed from a bed) get a penalty of 5000 to avoid recolouring loops. The crafting table is made first, since otherwise it eats the planks gathered for the recipe; ingredients are gathered in several passes until all are in the inventory at once. (3) Hunting for mob drops (leather, wool, string, pearls). (4) Mining: source blocks come from the game data (`drops`), the tool from `harvestTools`; a missing tool is obtained recursively. For deep ores not in reach it first digs a staircase down to the ore's usual level (diamonds -54) with three spare stone pickaxes, and if the stairs run into liquid it walks about 28 blocks away and starts again. Obsidian, rarely lying around, is made: water is poured onto the edge of a lava pool and the result is mined with a diamond pickaxe. Every step is logged and returned to the agent; if the player dies, the skill stops at once and reports where the items lie.

Instructions. `AGENTS.md` (goal, golden rules, how to play, memory) and seven guides (library, survival, programs and errors, recipes, building, teamwork, iron age): 7452 words in all. The guides were written from the pilots and tests; every piece of code in them was run on the server.

Agents. Each player is driven by its own Gemini 3.8 Flash High agent in the Antigravity CLI with all tools allowed. A watchdog (`agent-loop.sh`) runs turn after turn in the same conversation and adds the player's current status after each turn. Until 24 October two agents

played (alpha, beta), then four (gamma and delta joined at 17:22 UTC). The instructions state the Ender Dragon as the long-term goal and the current stage as numbered milestones.

Viewer. For watching we wrote a camera that follows the player from above, shows a cutaway underground (blocks above the player are not drawn) and darkens the sky with the game time (D13, D14). Agents do not see the viewer.

5 Audit method

The source of truth is the Minecraft server log. `extract.py` parses every log file into a table of events: advancements, deaths, joins and leaves, chat, and the feedback of console commands. Lines such as "Teleported ...", "Applied effect ..." or "Successfully filled ..." carry no player name; since the human spectator is the only operator, they come from the server console, which the supervising session used through `docker exec`.

The level of a milestone depends on the commands that appeared in the main run before it: **A** no console command at all; **B** only time changes and the `playersSleepingPercentage` rule (night skipping), and read-only queries; **C** assistance was active: status effects, enchantments, game rules (`keepInventory`), item gifts, block fills; **D** a teleport, damage, entity kill, item gift or advancement grant within 15 minutes before the milestone. Separately, **+K** counts the helpers the supervising session added to `team/autoload.js` during the run that the agents had already called (last column of Table 2); first use is taken from the program code. We could not attribute the definition of any of these helpers to a write by the agents themselves.

The main run starts at 2026-10-21 12:11 UTC, when the agents were launched. Programs at 12:06–12:10 were our library tests on the same player alpha and gave alpha a stone and an iron pickaxe before the start (Section 8); milestones M01 and M02 are therefore credited to beta, which reached them on its own.

Coverage. 3766 of the 3770 programs of the main run fall into stretches covered by the server log; the other 4 fall into seconds of server shutdown. The server log of 21 October 00:00–11:59 UTC is missing (rotation over restarts); this affects only the idle night after pilot 2, which we count from the bridge's chat log.

Active time is the union of program intervals of all agents, with gaps up to 5 minutes counted as play (the agent thinking); longer pauses (quota, stops) are excluded.

6 Results

6.1 Pilots

Pilot 1: two agents without guides or library, goal "build a structure". In 15 minutes: 8 deaths at night (34.9 per active hour) and about 20 wall blocks, after which the author stopped the run. Pilot 2, with guides and the first library: 2 deaths in active play, a 5×5 sandstone house (89 blocks, checked independently by both agents with `lib.materials`), and agents that declared "100 % COMPLETE" and ended their turns. The bots then stood without agents for 8.2 hours of night and died 42 times; about 60 of the 89 house blocks were left, most likely after creeper explosions. These two observations produced the turn watchdog (D15) and parking (D16).

6.2 Main run: milestones

Table 2 and Figure 1. The agents completed stage 1 at level A with no operator helpers: beta made the first stone tools 2 minutes after the start, diamonds and a diamond pickaxe after 2.0–2.1 active hours, obsidian (water on lava) after 2.4. Entering the Nether (delta, 7.4 active hours), the fortress (delta, 9.6) and the first blaze rod (gamma, 13.4) are level B, i.e. without

Table 1: Runs. Active hours: union of program intervals, gaps up to 5 min. lib %: share of programs calling the skill library.

run	agents	wall h	active h	programs	errors %	deaths	lib %
pilot 1 (no guides, no library)	2	0.2	0.2	99	16.2	8	0.0
pilot 2 (guides + library v1)	2	0.8	0.8	93	23.7	2	100.0
main run	4	294.0	20.1	3770	21.2	332	92.6

console help apart from night skipping, but with +K: the agents were already using operator helpers, among them `makeObsidianSafe` for the portal obsidian and `configureFireAvoidance` for walking around fire. Ender pearls and an eye of ender (beta, 18.9 and 19.1) are level C: permanent effects (Haste III, Resistance, Fire Resistance, Speed, Saturation), Looting III and `keepInventory` were active. We do not credit the agents with the stronghold portal room, entering the End and the dragon's death within 5 minutes (18:01–18:06 UTC on 2 November): the supervising session found the stronghold by reading the world's region files, filled the portal with `end_portal` blocks, teleported the players, killed the crystals, damaged the dragon with `/damage` and granted the advancement with `/advancement`: level D.

Table 2: Milestones of the main run: first time any agent reached each, with evidence level and the number of operator-added helpers already in use (+K).

id	milestone	first (UTC)	agent	wall h	active h	level	op. helpers
M01	Stone tools	2026-10-21 12:13	beta	0.0	0.0	A	0
M02	Iron pickaxe	2026-10-21 13:16	beta	1.1	1.0	A	0
M03	First iron armor piece	2026-10-21 13:17	beta	1.1	1.0	A	0
M04	Slept in a bed	2026-10-21 12:37	alpha	0.4	0.3	A	0
M05	Diamonds	2026-10-21 14:22	beta	2.2	2.0	A	0
M06	Diamond pickaxe	2026-10-21 14:24	beta	2.2	2.1	A	0
M07	Obsidian (water on lava)	2026-10-21 17:15	beta	5.1	2.4	A	0
M08	Flint and steel	2026-10-21 14:09	beta	2.0	1.9	A	0
M09	Entered the Nether	2026-10-24 17:53	delta	77.7	7.4	B	6
M10	Nether fortress	2026-10-25 21:02	delta	104.9	9.6	B	7
M11	Blaze rod	2026-10-31 15:21	gamma	243.2	13.4	B	8
M12	Ender pearl	2026-11-02 11:45	beta	287.6	18.9	C	14
M13	Eye of ender	2026-11-02 17:10	beta	293.0	19.1	C	16
M14	Stronghold portal room	2026-11-02 18:01	delta	293.8	19.9	D	16
M15	Entered the End	2026-11-02 18:01	alpha	293.8	19.9	D	16
M16	Ender dragon killed	2026-11-02 18:06	beta	293.9	20.0	D	16

6.3 Interventions

Table 3 and Figure 2. Phase A had no console commands. Phase B (from 23 October) had only 59 time and sleep commands (`time set`, `playersSleepingPercentage`) and 8 experience queries. Phase C began on 1 November at 17:00 UTC: after the author asked for "all improvements" (31 October), the supervising session extended `auto-skip-night.sh`, which switched on `keepInventory`, effects and enchantments through the console. The author started phase D on 2 November at 17:59 with the words "this is no longer the agents' work, it is dumb farming ... help them as much as possible". Phase C held 702 block fills (mostly rebuilding the portal frame every 15 s), 55 effects, 26 enchantments, 20 game rules, 15 spawn points and 2 item gifts. Phase D lasted 12 minutes: 15 teleports, 100 fills (End portal, walkways in the End), 5 damage

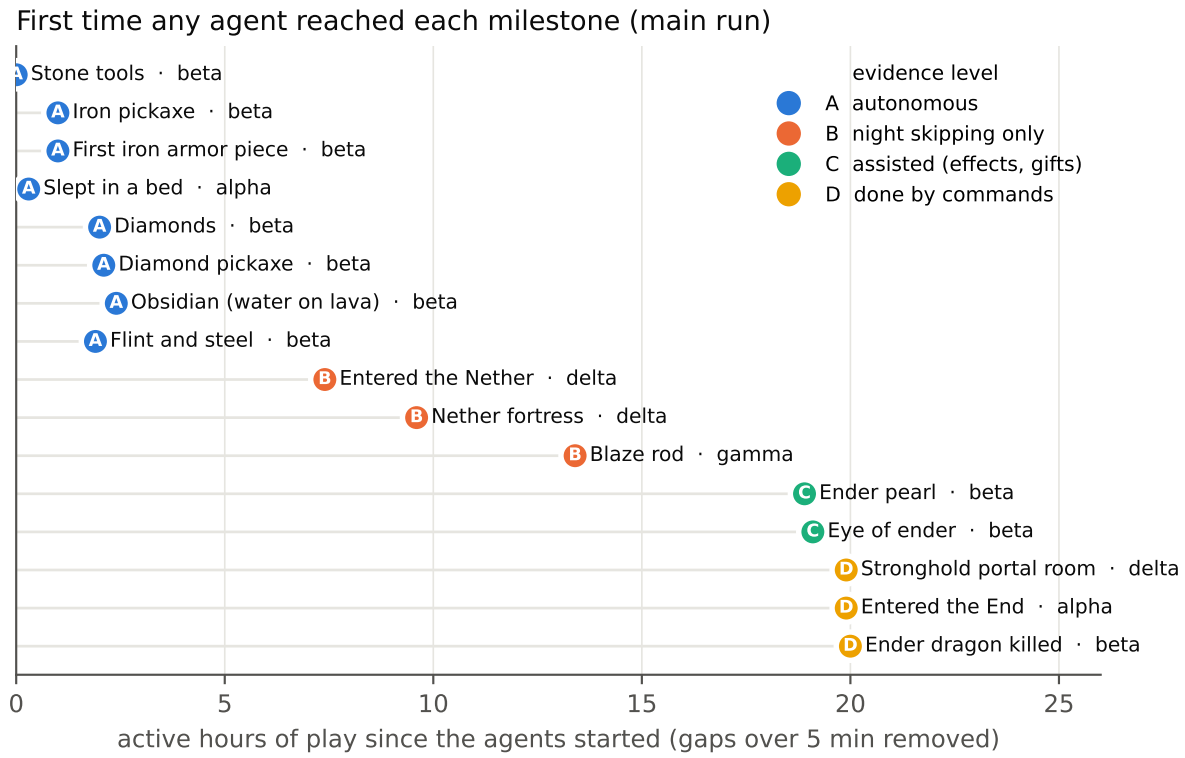


Figure 1: First time any agent reached each milestone, over active hours; colour and letter give the evidence level.

commands, 1 entity kill, 3 item gifts and 2 advancement grants; 1071 console commands in all.

6.4 Deaths

Table 4, Figures 3 and 4. Of the 332 deaths of the main run 211 (63.6 %) were fire, ghast fireballs and lava, i.e. the Nether. Phase A had 11 deaths in 4.5 active hours (2.4 per hour, mostly drowning), phase B 17.7 per hour, phase C 25.1. The rise coincides with the move into the Nether, not only with the assistance phase; even with Fire Resistance phase C produced 48 fire deaths as the agents walked carelessly over soul sand and near lava.

6.5 Program errors

Table 5. 798 of 3770 programs (21.2 %) failed; 70.4 % of the failures were time-outs, where the agent chose too short a time for a long action (mining, a march in the Nether). Errors in the code itself (TypeError, ReferenceError) are only 4.0 % of failed programs: the model writes JavaScript confidently; its weak point is judging how long actions and situations in the world take.

6.6 Agents and cost

Table 6. The median program runs 8.8 s. The share of programs calling `lib.obtain` is small (1.2–3.7 %) because the agents wrapped it in their own helpers; 92.6 % of programs use the library in some way. Tokens were counted from each agent's own stream (a model step is one response or tool call): 13 652 steps, 50.8 M uncached input tokens, 1922.9 M cached (97.4 %), 8.82 M output and 4.06 M reasoning tokens. The Antigravity quota forced pauses: from 27 to 30 October the game did not run at all (Figure 2).

Table 3: Console commands of the main run by family and phase.

family	A	B	C	D	total
fill	0	0	702	100	802
time	0	59	30	1	90
effect	0	0	55	21	76
enchant	0	0	26	0	26
gamerule	0	0	20	2	22
spawnpoint	0	0	15	0	15
teleport	0	0	0	15	15
query	0	8	0	4	12
give	0	0	2	3	5
damage	0	0	0	5	5
advancement grant	0	0	0	2	2
kill	0	0	0	1	1

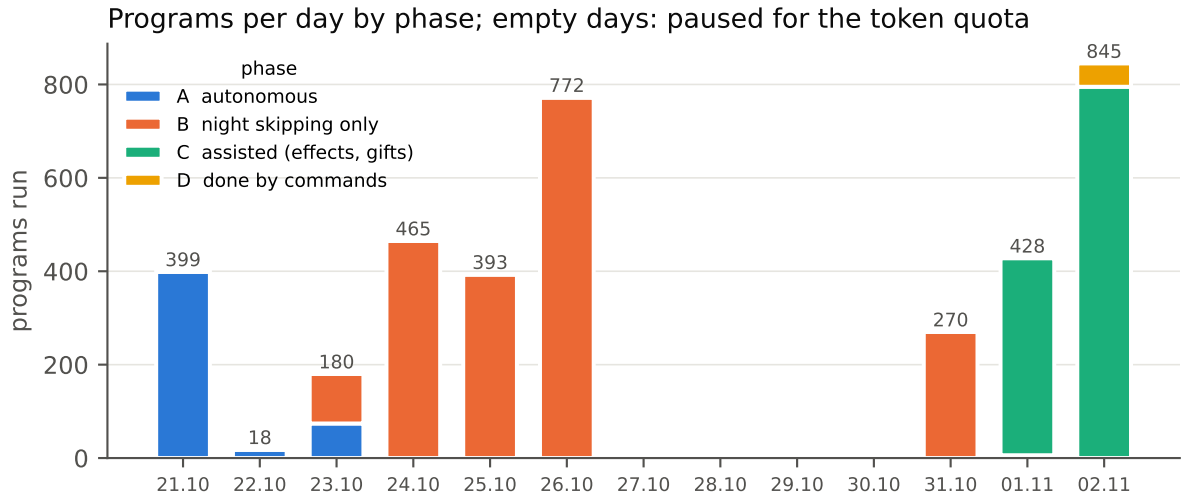


Figure 2: Programs per day by phase. Days without bars: the game was paused for the token quota.

6.7 Infrastructure defects

Table 7 lists twenty defects found while testing the skills and during the run. Some of them the agents would hardly have worked around on their own, since they lie in Mineflayer, its plugins or the viewer: a stack left on the cursor after a failed craft (D01), the path finder breaking walls (D03), doors it cannot pass (D04), air reported in ticks instead of bubbles (D11), mismatched palettes (D12). We consider this table a practical result in its own right for anyone building agents on Mineflayer.

6.8 Agent behaviour

We did not read all 3770 programs; below are episodes we watched live, each findable in the chat logs (UTC).

- *Agreement and self-organisation.* In pilot 1, 30 seconds after the start, beta proposed a wooden cabin and alpha agreed and named the site (19:00:57–19:01:27). In pilot 2 the proposals crossed: alpha accepted beta's plan (7×7) while beta accepted alpha's (5×5) (19:57:20–19:57:29); the agents noticed and settled on one site.

Table 4: Deaths of the main run by cause and phase.

cause	A	B	C	D	total
fire	0	54	48	0	102
ghast fireball	0	34	28	0	62
lava	1	23	23	0	47
drowning	8	18	15	2	43
arrow	2	18	8	0	28
fall or void	0	15	7	3	25
melee mob	0	8	9	3	20
explosion	0	1	2	0	3
other	0	2	0	0	2
total	11	173	140	8	332

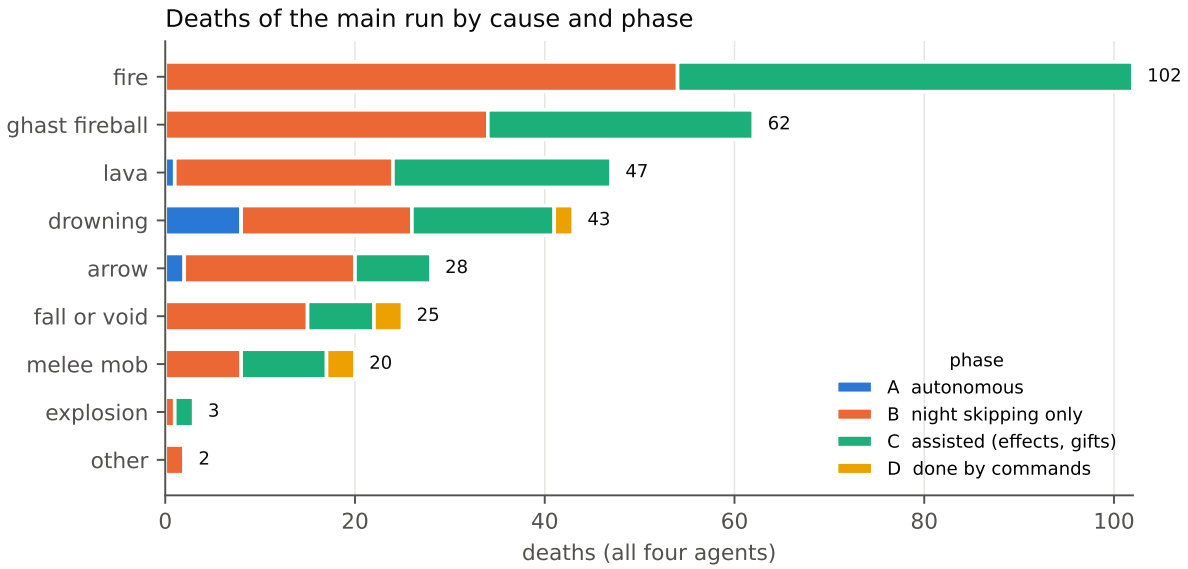


Figure 3: Deaths of the main run by cause, stacked by phase.

- *Adaptation.* Pilot 2 started in a desert with no tree within 128 blocks. The guides said nothing about sand; the agents decided to build from sandstone crafted from sand, and beta worked out the block budget of a 5×5 blueprint (20:04:43–20:07:52).
- *Care for the teammate.* "Beta: Skeleton in the pit at (36, 75, -19)! Stay above Y=78" (19:10:05, pilot 1); "Recovering items from creeper blast!" (19:08:58); in the main run, after alpha drowned, beta reported that the bed and spawn point were set and food and tools left in the chest, and asked where alpha's items were (21 October, 12:33:23).
- *Night protocol.* The agents waited out nights inside the base with long waiting programs (e.g. 21 October 12:17–12:35), as the guide asks, instead of working outdoors.
- *Ending the turn.* In pilot 2 both agents, having got "100 % COMPLETE" from `lib.materials`, wrote a summary and ended their turns although the instructions said to keep going; this is why the watchdog exists.
- *Reporting.* In phase C the agents repeated the same milestone in chat many times ("MILESTONE 8 100% COMPLETE!", 1 November 18:56–18:57, several agents in a row). At the end of the run the team notes contained the supervising session's entries about the stronghold and the dragon, which we do not credit to the agents.

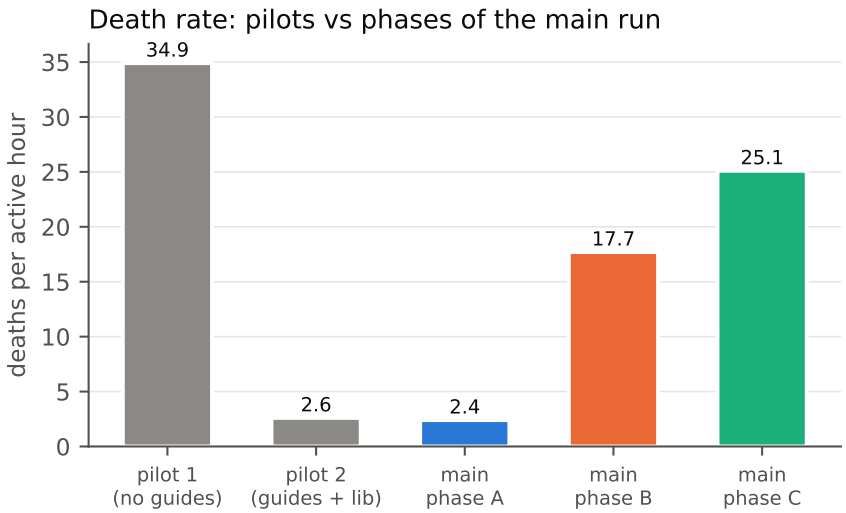


Figure 4: Deaths per active hour in the pilots and the phases of the main run (phase D lasted 12 minutes and is left out).

Table 5: Failed programs of the main run by error class.

class	programs	% of failed
timeout	562	70.4
other	83	10.4
movement	57	7.1
craft missing	44	5.5
type	21	2.6
reference	11	1.4
not found	10	1.3
place	3	0.4
bot offline	3	0.4
death	2	0.3
craft glitch	2	0.3

7 Comparison with earlier work

arXiv:2610.22319 [1]: four Gemini 4 Argon agents, no library, 27 h 14 min of continuous play, 605 deaths, dragon killed by the agents. Our run: 2–4 Gemini 3.8 Flash High agents with a library, 20.1 active hours over 12 days with quota pauses, 332 deaths; autonomous (A) only stage 1, the Nether up to the blaze rod at B with +K. Differences in model, budget and continuity do not allow attributing the gap to any one cause.

Odyssey [2]: a library of 223 skills, tasks up to diamonds and single fights; the library alone obtains a diamond within 10 minutes in 92.5 % of attempts. Our `lib.obtain` derives the chain for any item from the game data instead of 183 hand-written skills; the agents reached a diamond pickaxe in 2.1 active hours, but unlike Odyssey we have one run, not a series.

8 Limitations and threats to validity

One run. All numbers come from one main run and two short pilots; the spread is unknown.

Table 6: Agents of the main run. Token columns in millions; *cached* input was served from the context cache.

agent	first program	progr.	err. %	med. s	obtain %	deaths	steps	input	cached	output	think
alpha	2026-10-21 12:12	972	18.5	10.2	3.1	93	3770	16.9	524.5	2.48	1.14
beta	2026-10-21 12:11	1298	21.0	3.8	1.2	99	4376	14.57	634.7	2.62	1.2
gamma	2026-10-24 17:22	745	22.0	15.9	1.9	73	2645	9.22	353.6	1.86	0.87
delta	2026-10-24 17:22	755	24.0	11.5	3.7	67	2861	10.1	410.1	1.86	0.85

Alpha's head start. Our library tests at 12:06–12:10 gave alpha a stone and an iron pickaxe before the agents started. M01 and M02 are credited to beta, but the tools in the team may have sped up the early game.

Operator in the loop. During the run the supervising `agy` session changed `team/autoload.js` 10 times, the instruction template 7 times, `skills.js` 5 times, and the guides; these changes contained coordinates (blaze spawner, portal) and ready-made helpers. The +K mark reflects only helpers the agents called in code; it does not see advice in the instruction text.

Night skipping. We treat level B as nearly autonomous because skipping the night is equivalent to sleeping in a bed. But `playersSleepingPercentage = 1` and `time set` need no bed and so make the game easier than sleeping does.

Hidden help. From 1 November (19:24 UTC) the `logAdminCommands` rule was switched off and on 14 times; while it is off, commands an operator types in the game are not logged, while console commands still are. The human operator joined four times as a spectator; no command of theirs appears in the log.

Missing data. The server log of 21 October 00:00–11:59 UTC is missing; the idle night of pilot 2 is counted from the bridge log (42 deaths).

Changing library. `skills.js` and the guides changed during the run; versions are not pinned per milestone. We publish the repository state and its git history.

Model and access. Gemini 3.8 Flash High through Antigravity with a quota; other models were not tried. Token accounting for the pilots is missing.

Perception through an API. Mineflayer gives agents exact coordinates of blocks and entities, also behind walls; a human player has no such information.

9 Data and code availability

Research artifacts. The public repository `https://github.com/cyberdid/minecraft-ai-agents` contains the infrastructure code, the skill library, the agents' guides and instructions, the raw logs of the main run and both pilots (`logs/`, `runs/`), the frozen server log (`paper/data/raw/server-logs/`), saved worlds (`server/saves/`, including `stage1-complete-4agents-portal-lit` as a clean point before any assistance), the analysis scripts, generated tables and figures, and both PDFs. No software/data or preprint DOI has been assigned yet.

Table 7: Defects of the infrastructure found during testing and the run.

id	area	symptom	fix	found in
D01	crafting	A failed craft left the whole stack on the mouse cursor (36 planks lost)	stowCursor() after every craft attempt	test
D02	crafting	Result slot update never arrives on 1.20 (updateSlot timeout)	verify by inventory delta; shift-click the grid back; retry	pilot 1
D03	movement	Pathfinder dug through house walls and the floor under a door	shared list of placed blocks the pathfinder never breaks	test
D04	movement	Pathfinder cannot pass an open door	goThroughDoor(): open, walk with the movement keys, close	test
D05	placing	Placing against a chest opened the chest instead	prefer non-interactive neighbours; sneak otherwise	test
D06	gathering	collectBlock plugin hung for 200 s without a timeout	own collect(): per-block timeout, skip unreachable	test
D07	tools	Tunnelling wore out an iron, a stone and a wooden pickaxe	dig cost 6; spare pickaxes before a staircase; stop without a pickaxe	test
D08	food	Auto-eat ate the raw beef gathered for cooking	raw meat banned from auto-eat	test
D09	crafting	Recipe search chose 'recolour a black bed' and looped	penalty for recipes that consume the same item family	test
D10	inventory	Inventory view over-counted for a moment after pickups	re-check after a pause before crafting and before giving up	test
D11	survival	Bots drowned: air is reported in ticks (0-300), not bubbles	swim up below 60% of the air supply	main run
D12	viewer	3D view drew wrong blocks underground (block-state ids of 1.20.4 vs 1.20.1)	server moved to 1.20.1, the viewer's palette	main run
D13	viewer	Camera ended up inside rock (see-through block insides)	overhead camera with a cutaway of blocks above the player	main run
D14	viewer	Viewer always drew full daylight	sky and light dimmed from the game time	main run
D15	agents	Agents ended their turn after declaring the task done	watchdog continues the same conversation with fresh status	pilot 2
D16	agents	Idle bots died 42 times in one night and drew creepers to the base	parking: log out after 5 min without programs	pilot 2
D17	skills	obtain() silently restarted the whole chain after a death	abort on death and report where the items lie	test
D18	building	Portal frame without corners could not be placed	corner blocks added to the blueprint	test
D19	movement	Pathfinder used the cobblestone gathered for a recipe as scaffolding	cobblestone used as scaffolding only above 24	main run
D20	water	Water enclosed in rock cannot be scooped	only sources with an open surface	test

Reproduction. `paper/scripts/run_all.sh` rebuilds every table, number, figure and PDF from the raw logs in seven steps and logs each step with its exit code (`paper/release/run_all.log`). `paper/data/derived/manifest.csv` binds the results to the SHA-256 of every input file; `paper/release/SHA256SUMS` records the outputs and `paper/release/ENVIRONMENT.txt` the exact tool versions. Rebuilding does not repeat the agents' run itself: it recomputes every finding from its records.

Licensing. The code is licensed under the MIT License. Article text, original figures and experimental records are licensed under Creative Commons Attribution 4.0 International (`https://creativecommons.org/licenses/by/4.0/`), to the extent the author holds the relevant rights. Minecraft is a trademark of Mojang Synergies AB; this work is not affiliated with Mojang or Microsoft. The README illustration was generated by a Gemini image model.

Private data and release inspection. Before publication all new files were scanned for e-mail addresses, OAuth tokens, API keys and private keys; there were no matches. The logs contain only the in-game names of the agents and of the human spectator and the internal Docker network address. This check is not a universal privacy guarantee.

Models and software. Agents: `gemini-3.8-flash-high` through the Antigravity CLI (`agy` 1.4.x). Server: vanilla Minecraft 1.20.1 (`itzg/minecraft-server`). Mineflayer 4.39.0, mineflayer-pathfinder 2.4.5, prismarine-viewer 1.33.0, Node.js 22. Analysis: Python 3.12, matplotlib 3.11; PDFs built with Tectonic 0.15.

AI-assisted work. The infrastructure, skill library, agent guides, analysis scripts and a draft of this text were produced in Claude Code sessions (Claude Opus 5.5, Anthropic) directed by the author. The players were driven by Gemini 3.8 Flash High agents in the Antigravity CLI. A separate `agy` session (the supervising session) carried out the author's requests during the run: it started and stopped the game, edited the agents' instructions and helpers and, at the author's request, issued server console commands; these actions are disclosed in Sections 5, 6.2 and 8. The illustration was generated by a Gemini image model. The author is responsible for the study design, interpretations, references and final declarations. No AI system is an author.

10 Conclusion

Four low-cost coding agents with a tested skill library completed stage 1 of Minecraft autonomously in 2.4 active hours and, without console help other than night skipping, reached the first blaze rod in the Nether, although by then with helpers the operator had added. Later milestones were obtained under server-side assistance, and the end of the game was produced by operator commands; a mechanical audit of the server log separates these levels without manual judgement. The pace was limited by deaths in the Nether (63.6 % of all), program time-outs (70.4 % of failures) and the token quota rather than by the model's ability to write code. General conclusions need repeated runs, models compared under the same conditions, and a run without operator intervention up to the dragon.

11 History of revisions

v1.0 (2026-11-03): first version. While preparing it we corrected our own analysis errors: (1) the main run was first taken to start at the first program (12:06), which wrongly credited M01–M02 to alpha on the basis of our tests; (2) the archived pilot logs contained our library tests, so pilot 1 first showed 17 deaths instead of 8; (3) `gamerule playersSleepingPercentage` was first classified as assistance (level C) instead of night skipping (B).

References

[1] M. Faldor, S. Singh, R. Raileanu. A team of Gemini 4 Argon agents beats Minecraft. arXiv:2610.22319, 2026.

[2] S. Liu et al. Odyssey: Empowering Minecraft Agents with Open-World Skills. arXiv:2407.15325 (v3), 2025.

[3] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. TMLR, 2024. arXiv:2305.16291.

[4] L. Fan et al. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. NeurIPS Datasets and Benchmarks, 2022. arXiv:2206.08853.

[5] PrismarineJS. Mineflayer, v4.39. https://github.com/PrismarineJS/mineflayer.

[6] PrismarineJS. mineflayer-pathfinder and prismarine-viewer. https://github.com/PrismarineJS.

[7] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629.

[8] J. Liang et al. Code as Policies: Language Model Programs for Embodied Control. ICRA, 2023.